Systèmes d'exploitation (Charleroi)
Nouveaux appels système dans xv6

Pierre Vandenhove (pierre.vandenhove@umons.ac.be)
Gauvain Devillez (gauvain.devillez@umons.ac.be)

Année académique 2025–2026

Début : lundi 9 mars 2026
Remise : **mardi 7 avril 2026 à 17h**
Le but de ce projet est d'ajouter des appels système à `xv6`, un système d'exploitation minimaliste basé sur UNIX version 6 et développé par le MIT pour leur cours *6.1810 – Operating System Engineering*. Les objectifs de ce projet sont les suivants :

- mise en pratique des connaissances sur les processus et appels système ;
- modification d'un noyau de système d'exploitation (*kernel hacking*) ;
- comprendre du code existant et y contribuer.

1 Installation de xv6

Nous vous demandons d'utiliser la version de `xv6` développée pour l'architecture `RISC-V` (https://github.com/mit-pdos/xv6-riscv).[1] Compiler et exécuter cet OS n'est pas toujours trivial selon votre environnement de travail ; commencez le plus tôt possible. Vous pouvez télécharger le code source de `xv6` directement via `git` : par exemple, avec la commande

```
git clone https://github.com/mit-pdos/xv6-riscv .
```

Des instructions d'installation sont disponibles à l'adresse https://pdos.csail.mit.edu/6.828/2025/tools.html. Comme décrit dans ce lien, vous aurez besoin de l'émulateur `qemu` pour lancer `xv6`. Une fois toutes les dépendances téléchargées, vous pouvez compiler `xv6` en exécutant `make` dans le répertoire contenant le code source d'`xv6`. Vous pouvez ensuite le lancer avec `make qemu`. **Pour sortir de l'émulateur, vous pouvez utiliser la combinaison de touches `Ctrl-a` suivi de `x`.**

Le fonctionnement de `xv6` (RISC-V) est décrit dans le livre https://pdos.csail.mit.edu/6.828/2025/xv6/book-riscv-rev5.pdf ; il ne vous est évidemment pas nécessaire de lire ce livre en entier, mais vous pourriez y trouver des informations utiles (par exemple, les chapitres 2 et 4). Nous discuterons du code de `xv6` lié aux appels système à la séance de cours du lundi 9 mars 2026.

2 Ajout d'appels système

Vous ajouterez deux appels système au code de `xv6`.

Le premier est `int getreadcount()`, qui retourne le nombre de fois où l'appel système `read` a été appelé (et ce, peu importe si la lecture a fonctionné ou non). Le compteur (appelez-le `readcount`) est un compteur global à tous les processus. **Votre solution doit fonctionner même en cas d'appels concurrents à `read()` par plusieurs processus (cf. tests 2 et 3).** Pour ce faire, vous pouvez utiliser un `spinlock`, déjà implémenté dans `xv6`, comme expliqué au Chapitre 3 du cours.

Le second est `int getfilenum(int pid)`, qui retourne le nombre de **fichiers ouverts** par le processus dont l'identifiant est `pid`. En pratique, cela correspond au nombre de descripteurs de fichiers valides dans la table des fichiers ouverts du processus (incluant les trois fichiers standards : entrée, sortie, erreur). Si aucun processus avec l'identifiant `pid` n'existe, l'appel système doit retourner `-1`.

1. Il existe aussi une version pour l'architecture `x86` (https://github.com/mit-pdos/xv6-public), mais elle n'est plus maintenue. Si vous avez un ancien système et qu'installer les dépendances pour `RISC-V` s'avère compliqué, vous pouvez envisager d'utiliser cette version.

3 Conseils

Inspirez-vous des appels système existant qui réalisent des fonctions similaires. Ce projet requiert d'*écrire peu de lignes de code* ; la difficulté est de s'intégrer au code existant. Faites attention à suivre vos modifications au code de base de `xv6` (via `git` par exemple, ou via l'outil `diff`). Commencez par l'appel système `getreadcount()` et procédez par étapes ; par exemple, commencez par ajouter un appel système qui retourne `0`. Il est conseillé d'inclure la logique principale de la fonction `getreadcount()` dans le fichier `sysfile.c`, puisque ça concerne les fichiers (mais d'autres fichiers doivent également être modifiés). On vous laisse le soin de choisir où inclure la logique de `getfilenum()`. Pour comprendre l'organisation des fichiers ouverts par un processus, lisez le code des appels système liés à l'ouverture et la fermeture de fichiers. Vous n'avez pas accès à la librairie standard de C, mais plusieurs fonctions utiles sont déjà implémentées dans `xv6` ; par exemple, dans `kernel/string.c` pour manipuler des strings. Utilisez `grep` pour chercher des mots-clés dans le code source !

4 Tests

Un script bash avec quelques tests est disponible pour tester votre programme. Pour le lancer, les scripts de test doivent se trouver dans le même répertoire qu'un répertoire `src/` contenant le code d'`xv6`. Vous pouvez ensuite exécuter `bash test-xv6-system-calls.sh`. Le script commence par créer un fichier `Makefile.test` ; vous pouvez l'utiliser pour débugger avec `make -f Makefile.test qemu`. Lancer ces tests est la première chose que nous ferons pour évaluer votre projet.

Pour que les tests fonctionnent, les logiciels `gawk` et `expect` doivent être installés sur votre machine. Pour les installer sur une machine sous Ubuntu, vous pouvez exécuter la commande `sudo apt install gawk expect`.

Certains étudiants utilisant WSL ont parfois un problème avec le script en raison des caractères de fin de ligne (CRLF vs LF). Si c'est le cas, vous pouvez changer le caractère utilisé dans les tests, modifier le script, ou simplement lancer les tests à la main. Prévenez-nous si vous rencontrez un tel problème afin qu'on puisse adapter le script pour les années futures.

5 Modalités et remise

Le projet est à remettre sur Moodle pour le **mardi 7 avril 2026 à 17h** au plus tard. Vous devez remettre une archive `.zip` contenant le code modifié de `xv6` sur Moodle. Merci de rendre une archive contenant un seul dossier `src/` **ne contenant pas les fichiers compilés** ; lancez `make clean` avant de rendre votre projet. Ne rendez pas non plus les tests qu'on vous fournit. Incluez un fichier `README.txt` (à côté de `src/`) expliquant brièvement vos modifications.